ROTATE Engine: How to Run

A step-by-step guide for anyone on the team, no programming needed. Updated 6 Nov 2026 (engine 0.1.0, JSON contract 1.0, 188 tests).

1. What the engine is
2. Open a terminal
3. First-time install (new machine only)
4. The main commands
5. Giving the engine a farmer's choices
6. What-if, dry/wet years and a 4th card
7. Ready-made outputs (fixtures)
8. Check that everything works
9. Downloading fresh data (needs internet)
10. If something goes wrong
11. Rules when showing the output

1. What the engine is

The engine has **no screen or buttons**. It is a command-line program: you type a command, and it gives back its results as a **JSON file** (structured text). The app the team builds reads those files and draws the screens.

- **No internet needed** for any command in sections 4–8. All NASA data is already stored in the `data/` folder.
- Two sites are stored: `ctg_anwara` (Anwara, Chattogram: coastal salinity) and `barind_godagari` (Godagari, Rajshahi: Barind drought).
- The demo decision date is `2026-11-13`.

2. Open a terminal

In VS Code, press **Ctrl +** ` (the key above Tab). Then go to the project folder:

```
cd /home/sudipta/dev/NASA
```

Run every command in this guide from this folder. Every engine command starts with `.venv/bin/python -m rotate_engine`, which means "run the engine with the project's own Python".

3. First-time install (new machine only)

Skip this on Sudipta's machine: it is already installed. On a fresh clone, with Python 3.12:

```
python3.12 -m venv .venv
.venv/bin/pip install -e "engine[dev]"
```

If `python3.12 -m venv` is not available, the fallback with `uv venv --seed` is in `docs/INTEGRATION.md`. Then run the tests (section 8); they should all pass.

4. The main commands

Add `--out file.json` to save the result to a file (open it in VS Code to read it). Without `--out`, the result prints on the screen.

4.1 List the sites

```
.venv/bin/python -m rotate_engine sites
```

Prints both sites with their name, latitude, longitude and hazard.

4.2 Strategy cards (the main output)

```
.venv/bin/python -m rotate_engine cards --site ctg_anwara --date 2026-11-13 --out cards.json
```

Example result (Anwara, run on 6 Nov):

- Cards: Boro–Fallow–T.Aman (the farmer's current rotation) | Mungbean–Fallow–T.Aman | Wheat–Fallow–T.Aman
- Confidence: "In 4 of the 5 past seasons that started like this one, Boro peak (mid-March to mid-April) greenness in the area around this field (250 m) was above normal. Random years agree this much about 38% of the time. These are past examples, not a prediction for this season."

For the drought site, use `--site barind_godagari`.

4.3 Look-alike past years (analogs)

```
.venv/bin/python -m rotate_engine analogs --site ctg_anwara --date 2026-11-13 --out
analogs.json
```

The 5 past seasons whose 60 days before the decision date looked most like this year, and what happened afterwards in each.

4.4 Weather features table

```
.venv/bin/python -m rotate_engine features --site ctg_anwara --date 2026-11-13
```

Prints a table of each past season's window weather. It prints to the screen only.

4.5 Greenness curve (NDVI)

```
.venv/bin/python -m rotate_engine ndvi --site ctg_anwara --out ndvi.json
```

Add this	What it does
`--from 2024-11-01 --to 2025-05-31`	only that date range (here, one Boro season)
`--aqua`	adds points from a second MODIS satellite (Aqua), for display
`--window`	adds the spread of the 3×3 pixels around the field (Nov 2023 on)
`--hls`	the separate, sharper 30 m HLS curve (Nov 2015 on). Cannot be combined with `--aqua` or `--window`

4.6 One curve per past season

```
.venv/bin/python -m rotate_engine ndvi-years --site ctg_anwara --years 2003 2015 2019 --out
years.json
```

One Boro-season curve (1 Nov to 31 May) per harvest year. Use the look-alike years from section 4.3. `--aqua` is optional.

4.7 How well the method did in the past (backtest)

```
.venv/bin/python -m rotate_engine backtest --site ctg_anwara --date 2026-11-13 --out
backtest.json
```

Tests the method on every past season from 2001 to 2024. Its `verdict_text` currently says **"No demonstrable skill…"** at both sites, so the look-alike years are shown as historical examples only.

5. Giving the engine a farmer's choices

Create a file called `request.json` in the project folder:

```
{
  "current_rotation": "boro-fallow-t_aman",
  "salinity_class": "slight_4_8",
  "priorities": { "salinity": 0.7, "water": 0.0, "soil_n": 0.1, "yield_stability": 0.2 },
  "constraints": { "water_availability": "high" }
}
```

Then run:

```
.venv/bin/python -m rotate_engine cards --site ctg_anwara --date 2026-11-13 --input
request.json --out cards.json
```

Change the numbers and run again to see the cards change.

Field	Allowed values
`current_rotation`	a rotation ID from `rules/rules.json` (list below). It always becomes card 1
`salinity_class`	`non_saline_lt2`, `very_slight_2_4`, `slight_4_8`, `moderate_8_12`, `strong_12_16`, `very_strong_gt16`, `unknown`. This is local information, not NASA data
`priorities`	`water`, `soil_n` (soil nitrogen), `salinity`, `yield_stability`: each from 0 to 1. A higher number means it matters more to the farmer
`constraints.water_availability`	`low`, `medium` or `high`
`constraints.equipment`, `constraints.market`	free text (defaults `none` and `local`)

Some rotation IDs

ID	Rotation	ID	Rotation
`boro-fallow-t_aman`	Boro–Fallow–T.Aman	`wheat-fallow-t_aman`	Wheat–Fallow–T.Aman
`boro-fallow-fallow`	Boro–Fallow–Fallow	`mungbean-fallow-t_aman`	Mungbean–Fallow–T.Aman

`fallow-fallow-t_aman`	Fallow–Fallow–T.Aman	`maize-fallow-t_aman`	Maize–Fallow–T.Aman
`boro-aus-t_aman`	Boro–Aus–T.Aman	`lentil-fallow-t_aman`	Lentil–Fallow–T.Aman
`mustard-boro-t_aman`	Mustard–Boro–T.Aman	`soybean-fallow-t_aman`	Soybean–Fallow–T.Aman
`potato-boro-t_aman`	Potato–Boro–T.Aman	`boro_st-fallow-t_aman`	Boro (BRRI salt-tolerant variety)–Fallow–T.Aman

The full list (28 rotations) is under `patterns` in `rules/rules.json`.

6. What-if, dry/wet years and a 4th card

These are extra options. Add them to `request.json` next to the fields above.

What-if slider: "what if this season were drier and hotter?"

```
"mode": "scenario",
"shift": { "rain_pct": -50, "temp_c": 1.5, "rh_points": 0 }
```

`rain_pct` from −100 to 200 (percent change in rain), `temp_c` from −5 to 5 (°C added), `rh_points` from −30 to 30 (humidity points added). Only this season's 60-day window changes; past seasons stay as observed. Everything in the output is tagged **Scenario**.

Tercile scenario: "look only at past seasons that turned out dry"

```
"mode": "scenario",
"tercile": "dry"
```

`dry`, `normal` or `wet`. It can be combined with `shift`. All output is tagged **Scenario**.

A 4th card

```
"extra_card": true
```

Adds a fourth option to compare, if one with different dry-season crops exists.

7. Ready-made outputs (fixtures)

The `fixtures/` folder has real engine output for every demo case. You can open them without running anything, and the team builds the app against them.

File	What it is
`cards_ctg.json`	Anwara, salt class 4–8 dS/m, salinity first, water available
`cards_ctg_farm_a.json`	Anwara, same weather, no salt problem, water first (the "two farms, same weather" pair with `cards_ctg`)
`cards_barind.json`	Godagari drought, water first (the "two regions, two hazards" pair with `cards_ctg`)
`cards_ctg_high_salinity.json`	Anwara at 8–12 dS/m: a salt-tolerant option appears

`cards_barind_four.json`	Godagari with the 4th card
`cards_barind_whatif_dry.json`	what-if: half the rain, 1.5 °C warmer
`cards_ctg_tercile_dry.json`	tercile scenario: only past seasons that turned out dry
`ndvi_<site>.json`	full greenness curve, 2000–2026
`ndvi_<site>_extras.json`	with Aqua points and the 3×3 spread (Nov 2023 on)
`ndvi_<site>_hls.json`	the separate 30 m HLS curve (Nov 2015 on)
`ndvi_years_<site>.json`	season curves of the 5 look-alike years
`backtest_<site>.json`	the backtest result and verdict

The JSON shapes are described in `schemas/*.schema.json`.

8. Check that everything works

```
.venv/bin/python -m pytest -q engine
```

It should end with **188 passed** (about 16 seconds, no internet).

After changing data, rules or code (engine owner), rebuild the outputs, then rerun the tests:

```
.venv/bin/python scripts/build_fixtures.py
.venv/bin/python -m rotate_engine schemas --out-dir schemas
```

A test fails on purpose if the fixtures or schemas are out of date.

9. Downloading fresh data (needs internet)

Only scripts in `scripts/` use the internet; the engine itself never does.

Command	What it downloads	Login?
`.venv/bin/python scripts/fetch_power.py`	NASA POWER weather for every site (or name sites: `fetch_power.py ctg_anwara`)	No
`.venv/bin/python scripts/fetch_ndvi_ornl.py`	Aqua NDVI and the pixel window from ORNL	No
`.venv/bin/python scripts/fetch_ndvi_appeears.py <task_id>`	a finished AppEEARS NDVI task (add `hls` at the end for the HLS task)	Yes: `.env`
`.venv/bin/python scripts/submit_ndvi_hls.py`	submits a new HLS 30 m task; prints its task ID	Yes: `.env`

Event day: the stored weather stops on 2 Nov 2026, so refetch POWER before the demo. Then rebuild the fixtures and rerun the tests and the backtest (section 8).
Login: Earthdata username and password go only in the `.env` file, which is never committed. Never paste them into chat or code.

10. If something goes wrong

You see	What it means	Fix
`error: invalid input: …` (exit code 2)	a typo or a value outside the allowed range; the message names the field	check `request.json` against section 5
`"status": "not_cached"`	that site is not downloaded	use `ctg_anwara` or `barind_godagari`
`"status": "pending"` / "NDVI pending"	no greenness data for that site or date range	widen the dates, or download the NDVI data
`No such file or directory: .venv/bin/python`	you are not in the project folder, or it is not installed	`cd /home/sudipta/dev/NASA`, or follow section 3
`… is stale: run python scripts/build_fixtures.py`	a test found out-of-date fixtures	run the two commands in section 8

11. Rules when showing the output

- Say "compare" or "explore", never "best" or "you should plant".
- Never imply a forecast: the tool matches the weeks *before* the decision and shows past examples.
- Every card carries "needs local review". Every screen says the tool supports, not replaces, local agricultural expertise.
- NASA data shows observed conditions and vegetation, not how rotations performed. Rotation benefits come from the cited rule table. Keep them visibly separate.
- Show the backtest `verdict_text` word for word next to the look-alike years and the confidence.

Full details for the app builders: `docs/INTEGRATION.md`. Method, data sources and limits: `docs/ENGINE.md`.

ROTATE, NASA Space Apps Challenge 2026 (Chattogram), "Field Shift: Adapting Farms with NASA Data". Uses NASA data. Not endorsed by NASA.